

Vyzivovy modul

2-3 mesice

Open Food Facts · Database potravin · Vyzivove plany · Nakupni seznam · Mobilni app v1 · Tele. miry

Fitness & vyzivova platforma · Technicka dokumentace v1.0 · Duverny dokument

2025

Faze 2 stavi na funkci infrastruktury a autentizace z Faze 1. Cilem je kompletni vyzivovy modul – od databaze potravin az po mobilni aplikaci pro klienta. Vyzivovy poradce bude schopen vytvorit jidelnicek a klient ho uvidi v mobilni appce. Faze 2 je funkcně nejkomplexnější část projektu.

1 Database potravin

1.1 MongoDB kolekce foods

Navrhnout a vytvorit MongoDB schema pro potraviny

MongoDB

Schema

- Vytvorit kolekci foods v MongoDB s polem: externalId (UUID), name, source (system/custom), barcode
- Pole per100g: kcal, protein, carbs, fat, fiber, sugar, sodium – vsechny jako decimal
- Pole allergens[] – pole stringu (gluten, laktoza, orechy, vejce, soja, ryby...)
- Pole commonServings[] – { label: "1 ks", weightG: 120 } pro rychle zadavani mnozstvi
- Pole isVerified (bool) – odliši systemove potraviny od vlastnich poradce
- Pole nutritionistId (nullable) – vlastni potraviny jsou vazane na poradce
- Vytvorit MongoDB indexy: barcode (unique sparse), name (text index pro fulltextove hledani)
- Napsat seed script se 30–50 nejbeznejsimi potravinami jako startovaci data

Tip: Text index v MongoDB umoznuje fulltextove hledani pres pole name jednim dotazem bez extra knihovny.

1.2 Integrace Open Food Facts API

Napojeni na verejnou databazi potravin

Open Food Facts

HTTP Client

Cache

- Open Food Facts je bezplatne verejne API – zadna registrace ani API klic neni potreba
- Endpoint pro hledani: GET https://world.openfoodfacts.org/cgi/search.pl?search_terms={q}&json=1
- Endpoint pro ciarovy kod: GET <https://world.openfoodfacts.org/api/v0/product/{barcode}.json>
- Vytvorit IFoodExternalService interface a OpenFoodFactsService implementaci
- Pri hledani: nejprve hledat v lokalni MongoDB cache, az pak volat externi API
- Nalezene vysledky uložit do MongoDB s source="openfoodfacts" a isVerified=false
- Implementovat mapovaci vrstvu: OFF format -> interni Food model (normalizace jednotek)
- Pridat Polly retry policy: 3 pokusy s exponencialnim backoffem pri selhani OFF API
- Nastavit HttpClient timeout na 5 sekund – OFF muze byt pomale
- Cache platnost: 30 dni – po uplynutí znovu stahnout z OFF pro aktualni hodnoty

Pozor: OFF API nema SLA. Vzdy implementuj fallback na lokalni cache pri vypadu.

1.3 API endpointy - databaze potravin

REST endpointy pro praci s potravinami

REST API

Validate

Testy

- GET /foods/search?q=&category=&source= – fulltextove hledani, strankovat (page/pageSize)
- GET /foods/{foodId} – detail potraviny vctne vseh nutricnich hodnot
- GET /foods/barcode/{barcode} – hledani dle EAN/QR kodu (nejprve cache, pak OFF)
- POST /foods – vytvoreni vlastni potraviny (pouze role Nutritionist)
- PUT /foods/{foodId} – uprava vlastni potraviny (jen vlastnik)
- DELETE /foods/{foodId} – soft delete vlastni potraviny
- GET /foods/custom – seznam vlastnich potravin prihlaseneho poradce
- Pridat validaci: kcal musi odpovídat makrum ($\text{protein} \cdot 4 + \text{carbs} \cdot 4 + \text{fat} \cdot 9$) s toleranci 10%
- Napsat integracni testy pro search a barcode endpointy

2

Vyzivove plany - backend

2.1 MongoDB schema nutrition_plans

Datový model pro výživový plán

MongoDB

Denormalizace

Schema

- Dokument nutrition_plan: externalId (UUID), clientId, nutritionistId, name, status (draft/active/archived)
- globalSettings: { dailyKcal, proteinG, carbsG, fatG } – cíle klienta
- weeks[]: pole týdenních struktur – week.number, week.days[]
- days[]: dayOfWeek (1-7), meals[], dayTotals (vypočítané při uložení)
- meals[]: { name, time, foods[], mealTotals } – jídlo s potravinami
- foods[]: { foodId, foodName, amountG, kcal, protein, carbs, fat } – DENORMALIZOVANO
- Pole version (int) pro optimistické zamykání při souběžných úpravách
- dayTotals a mealTotals vypočítat vždy při POST/PUT – nikdy necítat z DB jako zdroj pravdy
- Přidat MongoDB index: clientId + status pro rychlé hledání aktivního plánu klienta

Tip: Denormalizace výživových hodnot do jídla je kličková – historické záznamy zůstanou správné i když se změní databáze potravin

2.2 CRUD endpointy pro výživové plány

Kompletní REST API pro správu plánu

REST API

MongoDB CRUD

- GET /nutrition/plans – seznam plánů poradce (filtr: clientId, status)
- POST /nutrition/plans – vytvoření nového plánu (status=draft, prázdné týdny)
- GET /nutrition/plans/{planId} – celý plán jako jeden dokument
- PUT /nutrition/plans/{planId} – úprava názvu, globalSettings (cíle maker)
- DELETE /nutrition/plans/{planId} – soft delete (status=archived)
- POST /nutrition/plans/{planId}/publish – změna status draft->active, klient plán uvidí
- POST /nutrition/plans/{planId}/duplicate – kopírování plánu jako šablony pro nového klienta
- PUT /nutrition/plans/{planId}/weeks/{w}/days/{d} – prepis celého dne (všechna jídla najednou)
- POST /nutrition/plans/{planId}/weeks/{w}/days/{d}/meals – přidání jídla do dne
- PUT /nutrition/plans/{planId}/weeks/{w}/days/{d}/meals/{mId} – úprava jídla
- DELETE /nutrition/plans/{planId}/weeks/{w}/days/{d}/meals/{mId} – smazání jídla
- POST /nutrition/plans/{planId}/weeks/{w}/days/{d}/meals/{mId}/foods – přidání potravin
- DELETE /nutrition/plans/{planId}/weeks/{w}/days/{d}/meals/{mId}/foods/{fId} – odebrání potravin

Pozor: Vždy overit, že nutritionistId v tokenu odpovídá vlastníkovvi plánu. Nikdy nesdílet plány mezi poradci bez explicitního souhlasu

2.3 Výpočet maker a validace

Automaticky vypocet nutrnicnich hodnot

Kalkulace

Unit testy

BMR/TDEE

- MacroCalculatorService: vypocitat dayTotals a mealTotals po kazde zmene
- Vypocet: $\text{kcal} = \text{protein} \cdot 4 + \text{carbs} \cdot 4 + \text{fat} \cdot 9$ (Atwater konverze)
- Po pridani/odebrani potraviny okamzite prepocitat mealTotals, dayTotals, weekTotals
- Endpoint pro navrh maker klienta: POST /nutrition/clients/{id}/calculate-goals
- Vypocet BMR: Mifflin-St Jeor rovnice (presnejsi nez Harris-Benedict)
- Vypocet TDEE: $\text{BMR} \cdot \text{activity factor}$ (1.2 / 1.375 / 1.55 / 1.725 / 1.9)
- Kaloricky deficit/surplus: -20% hubnutí, +10% nabírání, 0% udržení
- Rozdeleni maker: protein 30%, sacharidy 45%, tuky 25% (vychozi, upravitelne)
- Napsat unit testy pro MacroCalculatorService - vsechny vyjimkyne pripady

Tip: Mifflin-St Jeor: $\text{muži} = 10 \cdot \text{kg} + 6.25 \cdot \text{cm} - 5 \cdot \text{vek} + 5$, $\text{ženy} = 10 \cdot \text{kg} + 6.25 \cdot \text{cm} - 5 \cdot \text{vek} - 161$.

2.4 Nakupni seznam

Generator nakupniho seznamu z jidelnicku

Shopping list

Agregace

- GET /client/nutrition/plan/shopping-list?weekFrom=1&weekTo=2 - nakupni seznam
- Agregovat vsechny potraviny z vybraného rozsahu dni / tydnu
- Seskupit potraviny dle kategorie (ovoce, zelenina, maso, mlecne, obiloviny...)
- Secist mnozstvi stejne potraviny z ruznych jidel dohromady
- Vystup: { category, items: [{ name, totalAmountG, unit }] }
- Pridat volitelny parametr excludeDays[] - vynechat dny kde ma klient odlišný režim
- Napsat integracni test: plan se 3 tydny, overit spravne scitani a grupovani

3

Klientske endpointy

3.1 Dnešní jídelnicek a plan

API pro klientske zobrazeni jidelnicku

Klient API

Tracking

- GET /client/nutrition/plan/today - aktivni plan, dnesni den (vypocitany dle datumu)
- Logika: najit aktivni plan klienta, zjistit tyden a den v cyklu planu, vratit spravny den
- GET /client/nutrition/plan/week - cely aktualni tyden (pro mobilni prehled)
- GET /client/nutrition/plan/today/meals/{mealId} - detail jednoho jidla s potravinami
- POST /client/nutrition/log/meals/{mealId}/eaten - oznacit jidlo jako snezene
- GET /client/nutrition/log/today - co klient dnes snel (pro tracking maker)
- Pridat pole eatenAt (timestamp) ke kazdemu jidlu v daily logu
- Vypocitat zbyvajici makra dne: denni cil minus již snezene

Tip: Aktivni plan klienta zjistis dotazem: `nutrition_plans kde clientId=X AND status="active" ORDER BY publishedAt DESC LIMIT 1`.

3.2 Tělesna měření a hmotnost

Zaznam telesnych mer klienta

Mereni **PostgreSQL** **Statistiky**

- POST /client/measurements – uložit nové měření (váha, tuk, pas, boky, biceps, stehno, hrudník)
- GET /client/measurements – historie měření s filtrací datumu (from/to)
- GET /client/measurements/latest – poslední měření (pro zobrazení v profilu)
- GET /trainer/clients/{clientId}/measurements – poradce vidí měření svého klienta
- Validace: measuredAt nesmí být v budoucnosti, váha musí být 20–300 kg
- PostgreSQL tabulka BodyMeasurements už existuje z Faze 1 – použít ji
- Přidat statistiku: rozdíl oproti prvnímu měření, trend za posledních 30 dní
- Endpoint GET /client/measurements/stats – min, max, průměr, trend pro každou měřičku

3.3 Pokrok klienta pro poradce

Dashboard pokroku klienta

Dashboard **Compliance** **Analytics**

- GET /nutrition/clients/{clientId}/progress – souhrnný pokrok klienta
- Compliance score: počet splněných jídel / celkový počet jídel v plánu * 100
- Přidat streak počítací: počet po sobě jdoucích dní s compliance $\geq 80\%$
- Přidat týdenní přehled: průměrné kalorie, makra, compliance za posledních 7 dní
- GET /nutrition/clients/{clientId}/progress/weekly – týdenní detail pro grafy
- GET /trainer/clients/{clientId}/dashboard – kombinovaný dashboard (měření + compliance)
- Napsat integrační testy pro výpočet compliance a streaku

4 Webový portal – výživový modul

4.1 Database potravin (UI)

Stranka pro práci s potravinami ve webovém portalu

React **TanStack Table** **shadcn/ui**

- Stranka /foods – tabulka vlastních potravin s vyhledáváním (TanStack Table)
- Komponent FoodSearch – fulltext + filtr kategorie, zobrazení z OFF + vlastních
- Dialog AddFoodForm – formulář pro vytvoření vlastní potraviny (React Hook Form + Zod)
- Validace: kcal musí přibližně odpovídat makrum, povinné pole name a kcal
- Komponent NutritionBadge – malé barevné indikatory protein/sacharidy/tuky
- Zobrazit isVerified badge u systémových potravin vs "vlastní" u poradcových
- Infinite scroll nebo stránkování pro velké výsledky hledání

4.2 Editor výživového plánu

Hlavní editor pro tvorbu jídelníčku

Editor **Optimistic UI** **Auto-save**

- Stranka /plans/nutrition/{planId} – editor s týdenní pohledem (7 sloupců Po-Ne)
- DayColumn komponent – karta dne s jídly, makro sumou a progress bary
- MealCard komponent – rozbalovací karta jídla s tabulkou potravin
- AddFoodRow – inline přidávání potravin do jídla přes FoodSearch komponent
- Pole množství: editovatelné inline, při změně okamžitý přepočítávací maker (optimistic update)
- Tlačítko "Zkopírovat den" – zkopírovat všechna jídla z jednoho dne do jiného
- Sidebar s denními makry: progress bary kcal/protein/sacharidy/tuky vs. cíle klienta
- Tlačítko Publikovat – zavolat POST /publish, změna statusu na active, klient plan uvidí
- Upozornění při publikování pokud den překračuje kcal cíl o více než 15%
- Indikátor neuložených změn + auto-save po 2 sekundách nečinnosti (debounce)

Tip: Použij useMutation z TanStack Query s onMutate pro optimistic update – uživatel vidí změnu okamžitě bez čekání na API.

4.3 Cíle klienta a nastavení maker

Stranka pro nastavení výživových cílů klienta

Goals **BMR/TDEE** **Formular**

- Stranka /clients/{id}/nutrition-goals – formular pro nastavení cílů
- Sekce Anamnéza: věk, pohlaví, výška, váha, cíl (hubnutí/nabírání/udržení), aktivita
- Automaticky výpočet BMR a TDEE po zadání anamnézy (POST /calculate-goals)
- Zobrazit výpočet transparentně: BMR -> TDEE -> deficit/surplus -> výsledné kcal
- Makra: editovatelné posuvníky nebo +/- inputy, vizuální donut graf rozdělení
- Zobrazit distribuci jídel: 5 jídel s % z denních kalorií (snídaně 20%, svačina 10%...)
- Uložení cílů: PUT /nutrition/plans/{planId} s novými globalSettings

5 Mobilní aplikace - výživová část

5.1 Inicializace Expo projektu

Nastavení React Native aplikace

Expo SDK 52 **Expo Router** **MMKV**

- Vytvořit projekt: `npx create-expo-app mobile --template expo-router`
- Přidat závislosti: TanStack Query, Zustand, MMKV, Expo Notifications, Expo Camera
- Nakonfigurovat Expo Router – file-based routing v adresáři app/
- Struktura rout: (auth)/, (client)/, (trainer)/ – každá skupina má vlastní layout
- Zustand store: { user, accessToken, role } – uložit accessToken do MMKV (ne AsyncStorage)
- Nastavit Axios instance se stejnými interceptory jako web (auth token, refresh)
- Nakonfigurovat TanStack Query s MMKV jako persist cache storage
- Přidat app.config.js s environment variables (API_URL pro dev/prod)
- Otestovat spuštění na iOS simulator a Android emulator

Pozor: MMKV je 10x rychlejší než AsyncStorage. Použít pro všechna lokální uložení v aplikaci.

5.2 Onboarding a přihlášení

Auth flow v mobilní aplikaci

Auth **Deep link** **MMKV**

- Obrazovka (auth)/login – formulář email/heslo, volání POST /auth/login
- Obrazovka (auth)/invite/[token] – přijetí pozvánky přes deep link
- Po přihlášení uložit accessToken do Zustand + MMKV, refreshToken do MMKV
- Root layout (_layout.tsx) – kontrola přihlášení při startu, přesměrování
- Logout: smazat tokeny z MMKV, invalidovat všechny TanStack Query cache
- Ošetření expirace accessTokenu: interceptor volá /auth/refresh automaticky
- Deep link konfigurace pro pozvánkové URL (expo-linking)

5.3 Obrazovka Dnes – klient

Hlavní obrazovka klienta (client)/today

Today screen **SVG** **Offline**

- Kalorický kruh – SVG progress circle: splněné kcal vs. cíl, procento uprostřed
- Makro karty – protein, sacharidy, tuky: aktuální / cíl s progress barem
- Tydenní teaser – název dnešního tréninku s tlačítkem START (pokud existuje)
- Seznam jídel dne – 5 karet (snídane/svacina/obed/svacina/večere)
- Karta jídla: čas, název, kcal, stav (snezeno/na rade/naplanovano)
- Kliknutím na jídlo -> obrazovka detailu jídla
- Prefetch dat při otevření aplikace (queryClient.prefetchQuery)
- Pull-to-refresh pro aktualizaci denních dat
- Offline stav: zobrazit data z cache, indikátor "offline režim"

5.4 Detail jídla a označení jako snezené

Obrazovka (client)/nutrition/[mealId]

Meal detail **Optimistic**

- Zobrazit název jídla, čas, celkové kcal + makra (4 boxy)
- Tabulka potravin: název, množství, kcal, protein/sacharidy/tuky
- Poznámka poradce (pokud existuje) v barevném boxiku
- Tlačítko "Oznacit jako snezené" – POST /client/nutrition/log/meals/{id}/eaten
- Po označení: okamžitá aktualizace UI (optimistic update), refresh today screen
- Swipe-to-complete gesto jako alternativa k tlačítku
- Přidat možnost editace množství konkrétní potraviny (quick adjust)

5.5 Skenování čárového kodu

Expo Camera – skenování EAN kodu potravin

Barcode **Expo Camera** **Haptics**

- Použít Expo BarCodeScanner (součást Expo Camera v SDK 52)
- Obrazovka ScanBarcode – fullscreen kamera s overlay a hledacím rámečkem
- Po naskenování zavolat GET /foods/barcode/{barcode}
- Pokud potravina nalezena: zobrazit detail a tlačítko "Přidat do jídla"
- Pokud nenalezena: nabídnout ruční zadání nebo hledání v databázi
- Přidat povolení kamery do app.config.js (iOS NSCameraUsageDescription)
- Přidat haptic feedback při úspěšném naskenování (Expo Haptics)
- Ošetření: blikající kód, tmavé prostředí (fakeln)

Tip: Expo BarCodeScanner vrací event s type a data. Filtruj pouze CODE_128 a EAN_13 pro potraviny.

5.6 Tydenní jídelníček a nákupní seznam

Další obrazovky výživové sekce

Weekly view **Shopping list** **Share**

- Obrazovka (client)/nutrition/index – týdenní jídelníček, karty Po–Ne
- Kliknutí na den -> (client)/nutrition/[dayId] – detail dne, všechna jídla
- Makro summary pro každý den: progress barů kcal/protein/sacharidy/tuky
- Obrazovka (client)/nutrition/shopping-list – nákupní seznam z aktuálního týdne
- Nákupní seznam: grupovaný dle kategorie, checkbox pro skrtání nakoupených položek
- Checkbox stav uložit do MMKV (nechceme volat API při každém skrtnutí)
- Sdílení nákupního seznamu přes systémový Share dialog (Expo Sharing)

5.7 Záznam tělesných měr

Obrazovka (client)/progress/measurements

Measurements **ImagePicker** **Victory Native**

- Formulář pro zadání: váha (velký numerický pole s +/- tlačítky), 5 tělesných měr
- Foto sloty: 4 pozice (přední/bocní/zadní/volitelná) přes Expo ImagePicker
- Upload fotek do backendu: POST /client/photos s multipart/form-data
- Fotky uložit na Azure Blob Storage / MinIO (přes backend, ne přímo z mobilu)
- Zobrazit historii měření jako jednoduchý graf (Victory Native – linechart vahy)
- Porovnání: aktuální hodnota vs. první měření, barevný indikátor změny
- Napsat notifikaci 1x týdně: "Čas na měření!" (Expo Notifications)

Pozor: Fotky pokroku jsou citlivá data (GDPR). Endpoint pro upload musí overit identitu klienta.

6 Offline-first architektura

Offline režim pro vyzivova data v mobilní aplikaci

TanStack Query

MMKV persist

NetInfo

- Nastavit TanStack Query persistQueryClient s MMKV jako ulozistem
- staleTime: 5 minut pro vyzivova data (caste zmeny), 24 hodin pro plan klienta
- gcTime: 7 dni – data zustanou v MMKV i po zavreni aplikace
- Prefetch pri startu: today screen + aktualni tyden jidelnicku
- Offline detekce: pouzit @react-native-community/netinfo
- Pri offline stavu: zobrazit banner "Offline režim – zobrazuji ulozena data"
- Mutace pri offline: uložit do fronty v MMKV, odeslat pri obnoveni spojeni
- Označení jídla jako snezene musi fungovat offline (uložit lokalne, sync later)

Tip: persistQueryClient z balicku @tanstack/query-persist-client-core – pouzit s createAsyncStoragePersister upravenym pro MMKV

7 Vystupy Faze 2 – Definice hotovo

Vystup	Popis	Overeni
Databaze potravin	MongoDB schema, OFF integrace, vlastni potraviny	GET /foods/search vraci vysledky
Barcode lookup	Naskenuj EAN → potravina se najde nebo ulozi z OFF	GET /foods/barcode/{ean}
Vyzivovy plan CRUD	Poradce vytvori, edituje, publikuje plan	POST /publish zmeni status
Vypocet maker	Automaticky prepocet po kazde zmene potraviny	Unit testy MacroCalculator
Nakupni seznam	Agregace potravin z planu za zvoleny rozsah	GET /shopping-list vraci JSON
Klient API – dnes	Klient vidi spravny den dle aktualniho datumu	GET /today vraci korektni den
Telesna mereni	Klient zada mery, poradce je vidi, grafy fungují	POST /measurements ulozi do PG
Mobilni app – auth	Login, deep link pozvanka, token refresh	Prihlaseni na iOS i Android
Mobilni – today	Kaloricky kruh, makra, seznam jidel	Data z cache i pri offline
Mobilni – barcode	Kamera nasnima EAN, potravina se najde	Funguje na fyzickem zarizeni
Mobilni – mereni	Formular, upload fotek, graf vahy	Foto se objevi v Blob Storage
Web – editor planu	Poradce sestavi jidelnicek, vidi makra v realnem case	Publikovani dostupne klientovi

8 Doporucene poradí praci

Tyden	Priorita	Ukoly
1–2	Databaze potravin	MongoDB schema foods · Seed data · OFF integrace · /foods/search a /barcode endpointy
3–4	Vyzivove plany BE	nutrition_plans schema · CRUD endpointy · MacroCalculatorService · unit testy
5–6	Nakupni seznam + web	Generator nakupniho seznamu · Web databaze potravin · Editor planu v1
7–8	Klientske API	GET /today · Oznaceni jidla · Telesna mereni · Dashboard poradce
9–10	Mobilni app	Expo setup · Auth flow · Today screen · Meal detail · Barcode scanner
11–12	Mobilni – dalsi	Tydenny jidelnicek · Nakupni seznam · Mereni · Offline cache · Foto upload

Faze 2 je hotova tehdy, kdyz vyzivovy poradce muze sestavit jidelnicek ve webovem portalu a klient ho okamzite vidi v mobilni aplikaci – vctne funkcnio offline rezimu a skenování carových kodu.